

Arreglos dinámicos y estructuras

Programación I

UNRN

Universidad Nacional
de **Río Negro**

10

Septiembre

Programación I

Clase 10: Arreglos dinámicos y estructuras

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- **Declaración dinámica de arreglos.**
- **Problemas dimensionados en tiempo de ejecución.**
- **Declaración dinámica de matrices.**
- **Estructuras: Acceso a los componentes. Operaciones, creación dinámica. Sizeof. Uniones**

1 El Límite de los Arreglos Estáticos

¿Qué ocurre cuando no sabemos cuántos datos necesitaremos?

Arreglo Estático Tradicional

```
int notas[100];
```

- El tamaño se define obligatoriamente en tiempo de compilación.
- Vive en el **Stack** (memoria automática).
- Riesgo crítico: Si ingresan 101 elementos, ocurre un desbordamiento catastrófico (*Buffer Overflow*).

El Desafío en Tiempo de Ejecución

En problemas reales, el tamaño depende del usuario final:

- ¿Cuántos alumnos hay en el curso hoy?
- ¿Cuántos registros leeremos del archivo externo?
- Reservar espacio "de sobra" desperdicia RAM valiosa.

1 El Límite de los Arreglos Estáticos

Memoria Dinámica con malloc y free; El control manual del Heap en el lenguaje C

Asignación Dinámica: El Heap

Para dimensionar arreglos en tiempo de ejecución, acudimos a la librería `<stdlib.h>` y solicitamos memoria al sistema operativo bajo demanda.

malloc(): Reserva un bloque continuo de bytes sin limpiar.

sizeof(): Fundamental para calcular el espacio exacto multiplicando elementos por tipo.

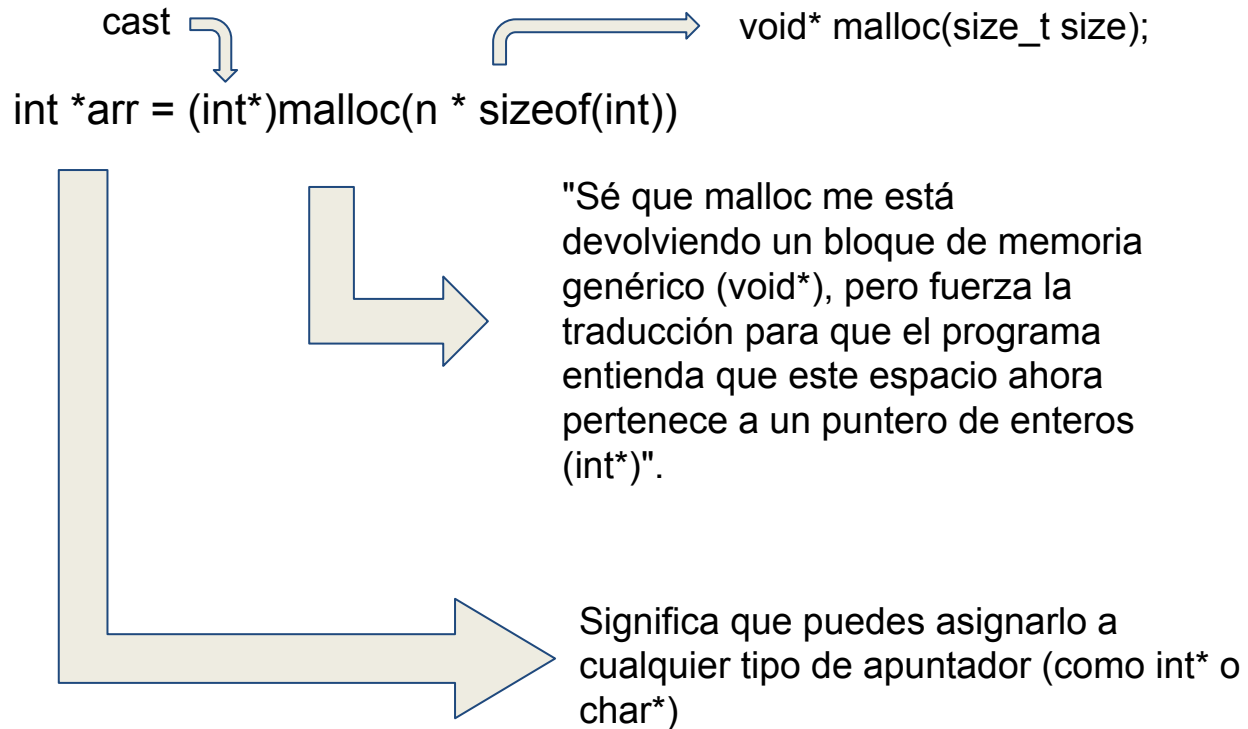
free(): Devuelve la memoria al sistema para evitar fugas (Memory Leaks).

```
#include <stdio.h>
#include <stdlib.h> // Necesaria para malloc y free

int main() {
    int cantidad;
    int *arreglo; //Puntero!
    printf("¿Cuántos numeros enteros quieres guardar?: "); //Consulta
    scanf("%d", &cantidad);
    arreglo = (int *)malloc(cantidad * sizeof(int)); // La reserva
    if (arreglo == NULL) {
        printf("Error: No se pudo asignar memoria.\n"); //Verifico espacio
        return 1; }
    for(int i = 0; i < cantidad; i++) {
        arreglo[i] = (i + 1) * 10; // Guardamos 10, 20, 30...
        printf("Elemento [%d]: %d\n", i, arreglo[i]); }
    free(arreglo);
    arreglo = NULL; // Evita punteros colgados
    return 0;
}
```

1 El Límite de los Arreglos Estáticos

Memoria Dinámica con malloc y free; El control manual del Heap en el lenguaje C



OJO: malloc puede no tener memoria para dar



```
if (arreglo == NULL) {  
    printf("Error: No se pudo  
asignar memoria.\n"); //Verifico  
espacio  
    return 1; }
```

1 Ejemplo Forzando el error

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 3;
    int *arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) return 1; // Validación de seguridad

    arr[0] = 10; arr[1] = 20; arr[2] = 30;
    printf("--- ZONA SEGURA (Acceso Legal) ---\n");
    printf("arr[0] = %d\n", arr[0]);
    printf("arr[2] = %d\n", arr[2]);
    printf("\n--- ZONA DE PELIGRO (Comportamiento Indefinido) ---\n");
    printf("arr[3] = %d <-- ¡Valor basura!\n", arr[3]);
    printf("arr[10] = %d <-- ¡Más basura!\n", arr[10]);

    printf("\nIntentando escribir en arr[50000]...\n");
    arr[50000] = 99; // ¡BAM! El Sistema Operativo nos descubre y aborta el programa.
    free(arr);
    return 0;
}
```

2 Declaración dinámica de matrices

Una declaración dinámica de matrices significa definir una estructura de datos de tamaño variable cuya memoria o dimensiones se asignan y ajustan en tiempo de ejecución, en lugar de tener un tamaño fijo predeterminado - lo que vimos antes, pero aplicada a **dos dimensiones**.

Comparativa: Vector vs. Matriz en C

- Vector Dinámico (1D): Pides un bloque continuo de memoria para tus elementos.

```
int *vector = (int *)malloc(columnas * sizeof(int));
```

- Matriz Dinámica (2D): Primero creas un vector dinámico que guardará direcciones de memoria (punteros), y luego a cada una de esas direcciones le asignas su propio vector dinámico de elementos.

```
int **matriz = (int **)malloc(filas * sizeof(int *)); // 1. Vector de punteros (filas), notar ** significa que apuntan a una dirección de memoria que contiene otro puntero!  
for (int i = 0; i < filas; i++)  
{  
    matriz[i] = (int *)malloc(columnas * sizeof(int));  
}
```

3 Problemas dimensionados en tiempo de ejecución.

```
#include <stdio.h>
#include <stdlib.h> // Necesario para usar malloc() y free()

int main() {
    int filas, columnas;
    printf("Introduce el número de filas: "); // Pedimos las dimensiones al usuario en tiempo de ejecución
    scanf("%d", &filas);
    printf("Introduce el número de columnas: ");
    scanf("%d", &columnas);

    // 1. Vector de punteros (filas)
    int **matriz = (int **)malloc(filas * sizeof(int *));

    // 2. A cada fila se le asigna su vector de elementos (columnas)
    for (int i = 0; i < filas; i++) {
        matriz[i] = (int *)malloc(columnas * sizeof(int));
    }
}
```

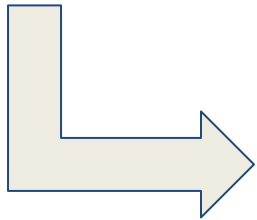
3 Problemas dimensionados en tiempo de ejecución. Cont

```
int contador = 1;
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        matriz[i][j] = contador++; // Guarda números consecutivos (1, 2, 3...)
    }
}
printf("\n--- Tu Matriz Dinámica ---\n"); //Imprimir
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        printf("%d\t", matriz[i][j]); // \t añade un espacio de tabulación
    }
    printf("\n"); // Salto de línea al terminar cada fila
}
for (int i = 0; i < filas; i++) {
    free(matriz[i]); // Liberamos de adentro hacia afuera: primero cada fila de enteros
}
free(matriz); // Finalmente, el vector que guardaba los punteros a las filas

printf("\nMemoria liberada correctamente.\n");
return 0;
}
```

4 Estructuras

Hasta ahora vimos en clases anteriores, un arreglo es una estructura que nos permite guardar múltiples valores, pero estrictamente del mismo tipo.



¿Qué pasa si queremos guardar los datos de un alumno?
Necesitamos texto para su nombre (char[]), enteros para su edad (int) y decimales para su promedio (float). La estructura (struct) nos permite agrupar diferentes tipos de datos bajo un solo nombre.

```
struct NombreEstructura {  
    tipo1 miembro1;  
    tipo2 miembro2;  
    // ...  
};
```



```
struct Alumno {  
    char nombre[50];  
    int edad;  
    float promedio;  
};  
struct Alumno alumno1;  
alumno1.edad = 20; // Acceso como L-Value (Escritura) con punto!
```

4 Estructuras: Ejemplo

```
#include <stdio.h>
#include <string.h> // Necesaria para usar strcpy

// Definición de la estructura
struct Alumno {
    char nombre[50];
    int edad;
    float promedio;
};

int main() {
    struct Alumno alumno1 = {"Cosme Fulanito", 20, 8.5}; // 1. Declaración e inicialización directa de una variable struct
    struct Alumno alumno2; // 2. Declaración y asignación posterior (miembro por miembro)

    strcpy(alumno2.nombre, "Ana Lopez"); // Para cadenas de texto (char array) se usa strcpy para asignar el valor (Cuidado aca!)
    alumno2.edad = 22;
    alumno2.promedio = 9.2;

    printf("--- Datos del Alumno 1 ---\n");
    printf("Nombre: %s\n", alumno1.nombre);
    printf("Edad: %d anos\n", alumno1.edad);
    printf("Promedio: %.2f\n", alumno1.promedio);

    return 0;
}
```

4 Estructuras: Ejemplo

```
#include <stdio.h>
#include <string.h> // Necesaria para usar strcpy

// Definición de la estructura
struct Alumno {
    char nombre[50];
    int edad;
    float promedio;
};

int main() {
    struct Alumno alumno1 = {"Cosme Fulanito", 20, 8.5}; // 1. Declaración e inicialización directa de una variable struct
    struct Alumno alumno2; // 2. Declaración y asignación posterior (miembro por miembro)

    strcpy(alumno2.nombre, "Ana Lopez"); // Para cadenas de texto (char array) se usa strcpy para asignar el valor (Cuidado aca!)
    // podria crear un puntero char *nombre; y ahora si alumno2.nombre = "Ana Lopez";
    alumno2.edad = 22;
    alumno2.promedio = 9.2;

    printf("--- Datos del Alumno 1 ---\n");
    printf("Nombre: %s\n", alumno1.nombre);
    printf("Edad: %d anos\n", alumno1.edad);
    printf("Promedio: %.2f\n\n", alumno1.promedio);

    return 0;
}
```

4 Estructuras:

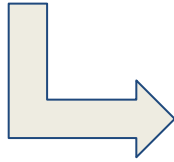
- La Gran Ventaja: Recordemos que C no permite la asignación masiva de arreglos completos de un solo golpe con el operador =. ¡Pero con las estructuras sí! Si tienes alumno1 y alumno2, puedes escribir alumno2 = alumno1; y C copiará mágicamente todos los campos de uno en el otro.
- No puedes sumar dos estructuras con + ni compararlas directamente con ==. Para saber si dos alumnos son iguales, debes comparar campo por campo (edad con edad, nombre con nombre usando strcmp).

Hacer ejercicio!: Diseñar un script que gestione la información de un producto utilizando estructuras (nombre, precio y cantidad). El programa debe permitir la lectura de datos por teclado y la posterior clonación de la información de una estructura a otra.

hint: scanf y fgets

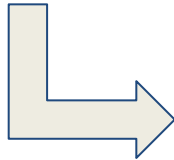
4 Estructuras: Acceso Directo vs Acceso mediante Punteros

Declarar una estructura estática normal (como una variable local).
Tienes el bloque de memoria física directamente en tu poder.



```
struct Alumno alumno1;  
alumno1.edad = 20; // Acceso directo
```

El Operador Flecha:



```
struct Persona *ptr = &p1; // ojo que estoy haciendo? lo que hago es al puntero pedirle que ahora  
apunte a la dirección donde está p1  
ptr->edad = 25; //pero ahora, en este caso, se accede con una "flecha"
```

El beneficio es esto (lo segundo) es que evito hacer esto:

```
// Un arreglo de 3 personas  
struct Persona grupo[3] = {  
    {"Jose", 25},  
    {"Juan", 30},  
    {"Jaime", 22}  
};
```



Proxima pagina

4 Estructuras: Acceso mediante Punteros

```
#include <stdio.h>
#include <stdlib.h> //
#include <string.h>
struct Persona {
    char nombre[50];
    int edad;
};

int main() {
    int cantidad = 3;
    struct Persona *grupo = (struct Persona *) malloc(cantidad *
sizeof(struct Persona)); //recuerdan el primer término?? (struct
Persona *)
    if (grupo == NULL) {printf("Error: No se pudo asignar
memoria.\n"); return 1; } //verificamos

//ahora asignamos:
strcpy(grupo[0].nombre, "Juan");
grupo[0].edad=10;
strcpy(grupo[1].nombre, "Jose");
grupo[1].edad=20;
strcpy(grupo[2].nombre, "Javier");
grupo[2].edad=30;
```

```
// Asignamos otro puntero para recorrerlo
struct Persona *ptr = grupo; //Sin & por que ya es un puntero

printf("--- Lista usando Memoria Dinamica ---\n");
for (int i = 0; i < cantidad; i++) {
    printf("Persona %d: %s, %d anos\n", i + 1, ptr->nombre, ptr->edad);
    ptr++; // Avanzamos al siguiente bloque de memoria
}

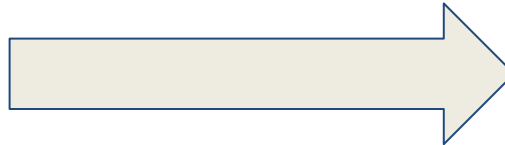
// Liberar memoria al finalizar
free(grupo);

return 0;
}
```

4 Estructuras: typedef

Cada vez que queremos crear una variable de nuestra estructura, C nos obliga a llamarla por su "nombre y apellido" completos (ej. struct Alumno alumno1;). Si tenemos que enviarla a funciones, crear punteros o arreglos, escribir la palabra struct 50 veces vuelve al código repetitivo. Podemos entonces usar typedef

```
struct Persona {  
    char nombre[50];  
    int edad;  
};  
  
int main() {  
    // Estamos obligados a repetir la  
    palabra 'struct'  
    struct Persona empleado1;  
    struct Persona empleado2;  
    return 0;  
}
```



```
// 1. Ponemos typedef al inicio  
typedef struct {  
    char nombre[50];  
    int edad;  
} Persona; // 2. Ponemos el "apodo" al final  
  
int main() {  
    // Ahora algo mas prolijo  
    Persona empleado1;  
    Persona empleado2;  
    return 0;  
}
```

4 Estructuras: Pasaje de estructuras a funciones por valor y por referencia (puntero)

Tendremos 2 opciones para el pasaje de variables a una función:

- Pasaje por Valor: la forma estándar, pero pesada, ya que la función recibe todas las variables
- Pasaje por Puntero: En lugar de copiar toda la estructura, le enviamos a la función únicamente la dirección de memoria donde está guardada la estructura original

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nombre[50];
    int edad;
} Paciente;

void mostrarPaciente(Paciente p) {
    printf("Paciente: %s, Edad: %d\n", p.nombre, p.edad);
    //solo podria mostrar, no modifico el original.
}

void cumplirAnios(Paciente *ptr_p) {
    ptr_p->edad = ptr_p->edad + 1; // Alteramos el dato real en la memoria
}
```

```
int main() {
    Paciente pac1;
    strcpy(pac1.nombre, "Ana");
    pac1.edad = 20;

    // 1. Pasamos por valor (Solo ponemos el nombre de la variable)
    printf("Antes del cumpleaños:\n");
    mostrarPaciente(pac1);

    // 2. Pasamos por puntero OJO!
    cumplirAnios(&pac1);

    // 3. Comprobamos el cambio
    printf("\nDespues del cumpleaños:\n");
    mostrarPaciente(pac1); // Ahora imprimirá 21

    return 0;
}
```

4 Estructuras: Uniones

Que es? un tipo de dato especial que permite almacenar diferentes tipos de variables en la misma ubicación de memoria.

Clave: En una estructura tradicional (struct), cada campo tiene su propio espacio de memoria reservado de forma independiente. En cambio, en una unión, todos los campos comparten exactamente el mismo espacio físico en la RAM.

El Tamaño (sizeof): Si usas (sizeof) para medir una **unión**, su tamaño se basará únicamente en su componente más grande. Ese es el espacio máximo que necesitará prestar a cualquiera de sus miembros para funcionar.

```
// 1. Una estructura tradicional
struct AlumnoStruct {
    char inicial;    // 1 byte
    int edad;       // 4 bytes
    float promedio; // 4 bytes
};
```



```
// 2. Una unión
union AlumnoUnion {
    char inicial;    // 1 byte
    int edad;       // 4 bytes
    float promedio; // 4 bytes
};
```

4 Estructuras: Uniones, ejemplo

```
#include <stdio.h>
```

```
// Declaramos nuestra unión  
union AlumnoUnion {  
    char inicial;    // 1 byte  
    int edad;        // 4 bytes  
    float promedio;  // 4 bytes  
};
```

```
int main() {  
    union AlumnoUnion alumno;  
  
    alumno.edad = 20;  
    printf("Paso 1 - Guardamos la edad.\n");  
    printf("Edad actual: %d\n\n", alumno.edad);  
  
    alumno.promedio = 8.5;  
    printf("Paso 2 - Guardamos el promedio.\n");  
    printf("Promedio actual: %.2f\n\n", alumno.promedio);  
  
    printf("Paso 3 - Intentamos leer la edad original otra vez...\n");  
    printf("Edad arruinada: %d\n", alumno.edad);  
  
    return 0;  
}
```



Por esto mismo, las uniones (union) son súper eficientes para ahorrar memoria, pero exigen que el programador lleve un control muy estricto de qué dato fue el último que se guardó